

Q1)

```
1  def MaximumProfit(arr):
2
3      maximumSum = arr[0]
4      currentSum = arr[0]
5
6      for i in range(1, len(arr)):
7          currentSum = currentSum + arr[i]
8          currentSum = max(arr[i], currentSum)
9          maximumSum = max(maximumSum, currentSum)
10
11     return maximumSum
12
```

This is a largest subarray sum problem. We use the first elements of the array as our baseline and calculate it using them. This way it becomes a dynamic programming question. Since we have only one for loop with basic operations, time complexity is $O(n)$.

The previous implementation of this was without dynamic programming. It was a recursive method, with $O(n.\log n)$ time complexity. So, dynamic programming makes it faster. There is no recurrence relation here, because I chose to implement it in a non-recursive way, using dynamic programming, called memorization technique.

$$\sum_{i=1}^n 1 = n \rightarrow T(n) = O(n)$$

Q2)

```
def CandyShop(capacity, lengths, prices):
    size = len(prices)
    #create an double array[][] filled with zeroes.
    array = [[0 for x in range(capacity + 1)] for x in range(size + 1)]

    for i in range(size + 1):
        for j in range(capacity + 1):
            if (i == 0 or j == 0):
                array[i][j] = 0
            elif lengths[i-1] <= j:
                array[i][j] = max(prices[i-1]
                                   + array[i-1][j-lengths[i-1]],
                                   array[i-1][j])
            else:
                array[i][j] = array[i-1][j]

    return array[size][capacity]
```

This is a very similar problem to Knapsack problem. We have the prices instead of values; and lengths instead of weights. In this problem however, we don't have a specified knapsack size already known. What we have though, is the length of the prices array; which becomes our knapsack size.

We must choose some numbers such that they are the most profitable choice.

Time complexity for this problem is calculated as $O(\text{size} \times \text{capacity})$ which might be closer to $O(n^2)$ in some cases. The reason behind this, is there are 2 for loops with ranges size and capacity. I also did not choose to do this in a recursive way and opted for an iterative solution.

$$\sum_{i=1}^n \sum_{j=1}^m 1 = \sum_{i=1}^n n \rightarrow T(n) = O(n.m)$$

Q3)

```
def FractionalKnapsack(weight, price, capacity):  
    packs = []  
    size = len(weight)  
    for i in range(size):  
        packs.append(Specs(weight[i], price[i]))  
  
    packs.sort(reverse=True)  
  
    result = 0  
  
    for i in packs:  
        currentWeight = i.weight  
        currentPrice = i.value  
  
        if (capacity - currentWeight >= 0):  
            capacity = capacity - currentWeight  
            result = result + currentPrice  
        else:  
            fraction = capacity / currentWeight  
            result = result + (currentPrice * fraction)  
            capacity = capacity - (currentWeight * fraction)  
            break  
    return result
```

This is a Fractional Knapsack problem. We are sending two arrays, and do some basic stuff with them. First of all, sorting decreasingly. Python uses Timsort when `sort()` is called, which is $O(n \cdot \log n)$ time. Then, we go to the for loop, which runs $O(n)$ time. Inside of the for loop are all fundamental operations and only take $O(1)$ time.

Thus, the whole function is $O(n \cdot \log n)$ time. But there is a catch. If the arrays are already sorted in reverse order, we would not have to sort them, which would render the time complexity as $O(n)$ time.

Explanation of the algorithm: Greedy approach to this problem, is by calculating the ratio between price/weight. We sort the arrays using this ratio, and try to add it to the result. Then, take the next largest ratio, and add that one as well. Do this until there is no more capacity left. If we cannot add the last element fully; take only some part of it; i.e. fraction of it.

Q4)

```
def CourseSelection(course, start, finish):  
    n = len(start)  
    array = [0]*n  
    array[0] = course[0]  
  
    i = 0  
    count = 0  
  
    for j in range(n):  
        if(start[j] >= finish[i]):  
            count = count+1  
            array[count] = course[j]  
            i = j  
    return count, array
```

This is an Activity Selection problem. In a given time frame, we must select non-conflicting activities, which are marked with a start and finish time.

For this to be greedy, we are selecting the first element of the courses as our baseline. We need to keep track of the last element to see which element is going to start next after this activity finishes; with $i=0$.

When we compare the last selected activity with the current activity in the if statement, we append this to the array with $\text{array}[\text{count}] = \text{course}[\text{j}]$. And lastly, we return the count and the array.

Since there is only one for loop with basic statements, it is $O(n)$ time. However, if the array we pass is not already sorted; then we would have to sort it, which would make it $O(n \log n)$

When the array given, is already sorted:

$$\sum_{i=1}^n 1 = n \rightarrow T(n) = O(n)$$